LANGCHAIN AGENTS & MEMORY - COMPLETE GUIDE FOR BEGINNERS

Understanding Agents, Tools, Memory, and Advanced Concepts
Built on top of what you already learned!

🎯 WHAT YOU'LL LEARN

- ☑ What are Agents and why they're powerful
- ☑ How Agents differ from Chains
- ☑ Types of Agents and when to use them
- ☑ Tools: How Agents interact with the world
- ☑ Memory: Making AI remember conversations
- ☑ Complete hands-on projects

CHAPTER 1: UNDERSTANDING AGENTS - The Smart Decision Makers

🤖 What is an Agent?

Simple Definition: An **Agent** is an AI that can **THINK**, **DECIDE**, and **USE TOOLS** to accomplish a task.

🔄 Chain vs Agent (Critical Difference!)

CHAIN (What you learned before):

```
You → Prompt → LLM → Response
    (Fixed path, no decisions)
```

AGENT (New concept!):

```
You → Question → Agent → Thinks → Decides what to do
                    ↓
            Uses Tools (calculator, search, database)
                    ↓
            Checks result → Decides next step
                    ↓
            Final Answer
```

Real-World Analogy

Chain = Recipe:

- Step 1: Boil water

- Step 2: Add pasta
- Step 3: Cook 10 minutes
- *Always the same steps, no thinking*

Agent = Chef:

- Goal: Make dinner
- *Thinks:* "What ingredients do I have?"
- *Decides:* "I'll make pasta"
- *Uses tools:* Checks pantry, uses stove
- *Adjusts:* "No pasta? I'll make rice instead"
- *Flexible and adaptive!*

Visual Comparison

```
┌─────────────────────────────────────────────────────────┐
│                    CHAIN                                │
├─────────────────────────────────────────────────────────┤
│ Input → Template → LLM → Output                         │
│ (Linear, predictable, no decisions)                     │
└─────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────┐
│                    AGENT                                │
├─────────────────────────────────────────────────────────┤
│ Input → Agent Brain → [Thought Process]                 │
│              ↓                                          │
│         Decision: Use Tool A or Tool B?                 │
│              ↓                                          │
│         Execute → Check Result → Decide Next Step       │
│              ↓                                          │
│         Final Answer                                    │
│ (Dynamic, adaptive, makes decisions)                    │
└─────────────────────────────────────────────────────────┘
```

When to Use Agents vs Chains

Use CHAINS when:

- Task is straightforward and predictable
- Steps are always the same
- Example: "Translate this text to Spanish"

Use AGENTS when:

- Task requires multiple steps
- Steps depend on intermediate results
- Need to use external tools
- Example: "Research a topic and create a summary with current data"

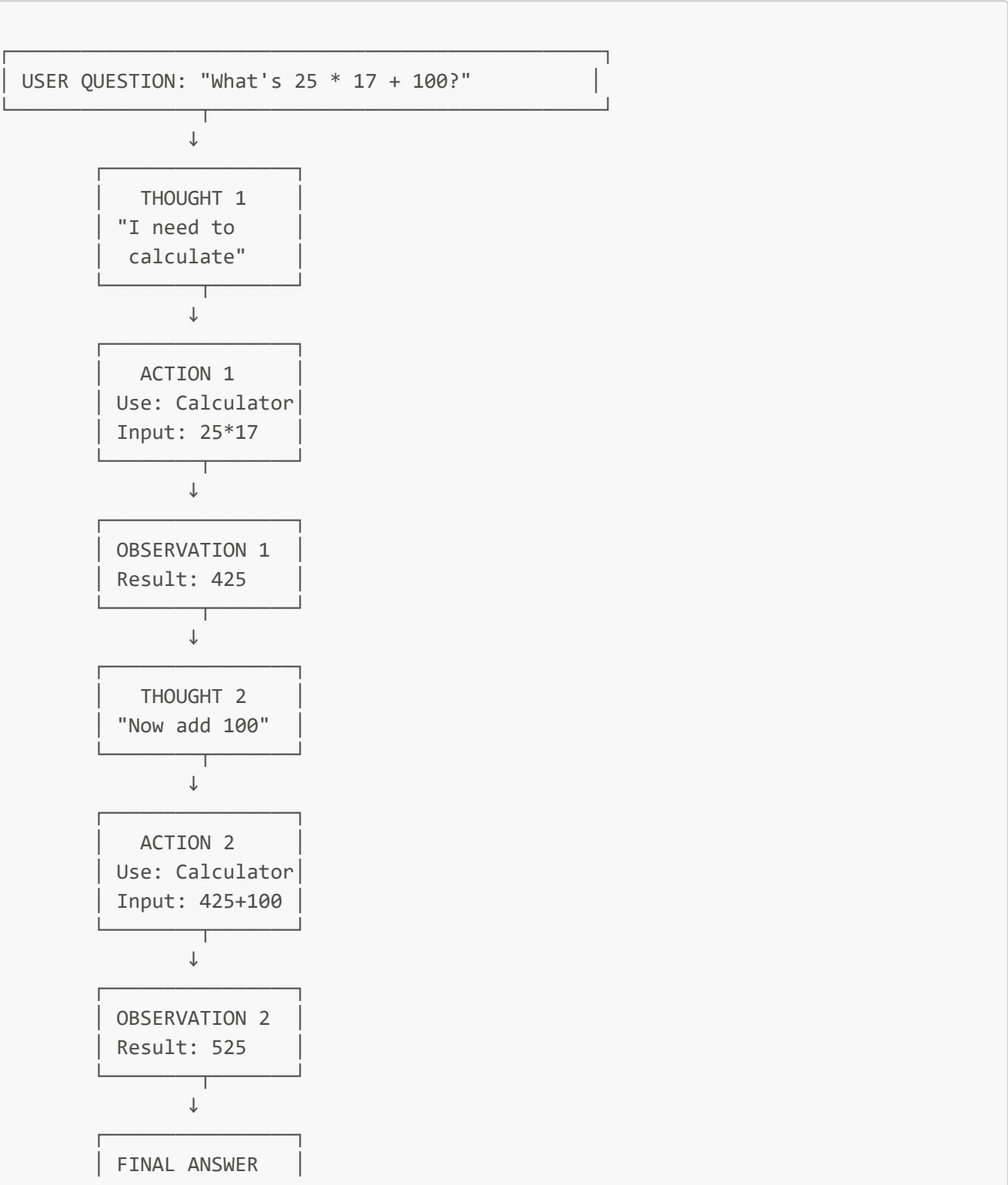
CHAPTER 2: HOW AGENTS WORK - The ReAct Framework

🧠 The ReAct Pattern (Reasoning + Acting)

Agents use a pattern called **ReAct** (Reason + Act):

1. **Thought** - Agent thinks about what to do
2. **Action** - Agent decides which tool to use
3. **Observation** - Agent sees the result
4. **Repeat** - Until task is complete

The Agent Loop (Visual)



```
          │ "The answer    │
          │  is 525"       │
          └────────────────┘
```

Real ReAct Example Output

```
Question: What's the weather in Paris and should I bring an umbrella?

Thought: I need to find the current weather in Paris
Action: weather_tool
Action Input: Paris
Observation: Current weather: Rainy, 15°C, 80% chance of rain

Thought: It's rainy, so an umbrella is recommended
Final Answer: The weather in Paris is rainy (15°C). Yes, you should
definitely bring an umbrella as there's an 80% chance of rain.
```

CHAPTER 3: TOOLS - Giving Agents Superpowers

🛠️ What are Tools?

Tools are functions/capabilities that agents can use. Think of them as the agent's **hands** to interact with the world.

Common Tool Types

```
┌──────────────────────────────────────────────────┐
│              AGENT TOOLS                         │
├──────────────────────────────────────────────────┤
│ 🔍 Search Tools                                  │
│    • Web search (Google, DuckDuckGo)             │
│    • Wikipedia search                            │
│                                                  │
│ 🧮 Calculation Tools                             │
│    • Calculator (math operations)                │
│    • Python REPL (run code)                      │
│                                                  │
│ 📊 Data Tools                                    │
│    • SQL database queries                        │
│    • CSV/Excel readers                           │
│    • API calls                                   │
│                                                  │
│ 📝 Action Tools                                  │
│    • Send emails                                 │
│    • Write files                                 │
│    • Make API requests                           │
└──────────────────────────────────────────────────┘
```

Anatomy of a Tool

Every tool has 3 parts:

```
Tool(
    name="calculator",              # 1. NAME: What it's called
    description="Useful for math",  # 2. DESCRIPTION: When to use it
    func=calculate_function         # 3. FUNCTION: What it does
)
```

EXAMPLE 1: Creating a Simple Custom Tool

```
# Installation
!pip install -q langchain langchain-google-genai google-generativeai langchain-
community

# Imports
from langchain.agents import Tool, AgentExecutor, create_react_agent
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain import hub
import os

# Setup API
os.environ["GOOGLE_API_KEY"] = "YOUR_API_KEY_HERE"

# Create LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0
)

# Step 1: Define a simple tool function
def get_word_length(word: str) -> str:
    """Returns the length of a word"""
    return f"The word '{word}' has {len(word)} letters."

# Step 2: Create a Tool object
word_length_tool = Tool(
    name="WordLength",
    func=get_word_length,
    description="Useful when you need to find how many letters are in a word.
Input should be a single word."
)

# Step 3: Test the tool directly
print(word_length_tool.run("elephant"))
# Output: The word 'elephant' has 8 letters.
```

EXAMPLE 2: Multiple Tools - Calculator & Word Counter

```
# Tool 1: Calculator
from langchain.tools import Tool

def calculator(expression: str) -> str:
    """Evaluates a mathematical expression"""
    try:
        result = eval(expression)
        return f"The result is: {result}"
    except:
        return "Invalid math expression"

calc_tool = Tool(
    name="Calculator",
    func=calculator,
    description="Useful for mathematical calculations. Input should be a math
expression like '25 * 4 + 10'"
)

# Tool 2: Word Counter
def count_words(text: str) -> str:
    """Counts words in a sentence"""
    word_count = len(text.split())
    return f"The text has {word_count} words."

word_counter_tool = Tool(
    name="WordCounter",
    func=count_words,
    description="Counts the number of words in a text. Input should be a sentence
or paragraph."
)

# Test both tools
print(calc_tool.run("100 + 50 * 2"))
print(word_counter_tool.run("The quick brown fox jumps"))
```

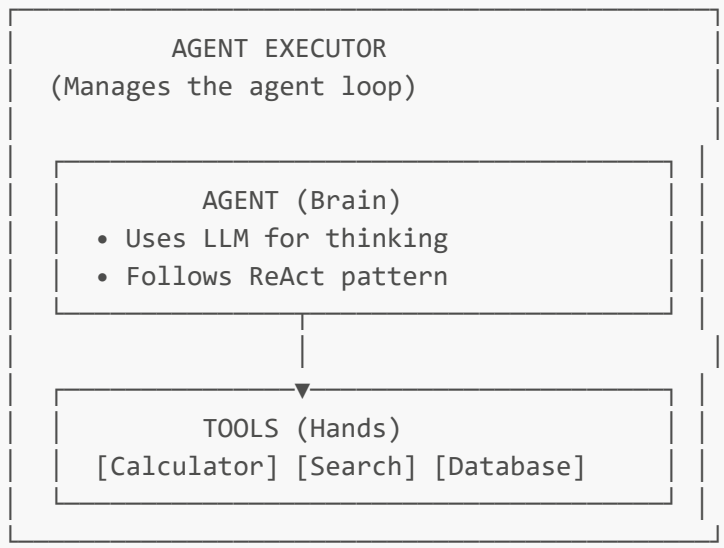
CHAPTER 4: CREATING YOUR FIRST AGENT

🎬 Agent Components

To create an agent, you need:

1. **LLM** - The brain (AI model)
2. **Tools** - Capabilities (what agent can do)
3. **Agent Type** - How agent thinks (ReAct, Zero-shot, etc.)
4. **AgentExecutor** - Runs the agent loop

Agent Architecture



EXAMPLE 3: Building Your First Agent

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.tools import Tool

# Step 1: Setup LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0  # Low temperature for focused reasoning
)

# Step 2: Create tools
def multiply(expression: str) -> str:
    """Multiplies two numbers. Input: 'number1 * number2'"""
    try:
        nums = expression.replace('*', ',').split(',')
        result = float(nums[0].strip()) * float(nums[1].strip())
        return f"{result}"
    except:
        return "Please provide format: number1 * number2"

def add(expression: str) -> str:
    """Adds two numbers. Input: 'number1 + number2'"""
    try:
        nums = expression.replace('+', ',').split(',')
        result = float(nums[0].strip()) + float(nums[1].strip())
        return f"{result}"
    except:
        return "Please provide format: number1 + number2"

tools = [
    Tool(
```

```
        name="Multiply",
        func=multiply,
        description="Multiplies two numbers. Use this when you need to multiply.
Input format: 'number1 * number2'"
    ),
    Tool(
        name="Add",
        func=add,
        description="Adds two numbers. Use this when you need to add. Input
format: 'number1 + number2'"
    )
]

# Step 3: Get prompt template from LangChain Hub
# This template tells the agent HOW to think (ReAct pattern)
prompt = hub.pull("hwchase17/react")

# Step 4: Create agent
agent = create_react_agent(
    llm=llm,
    tools=tools,
    prompt=prompt
)

# Step 5: Create agent executor (this runs the agent loop)
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,  # Shows thinking process
    handle_parsing_errors=True
)

# Step 6: Run the agent
response = agent_executor.invoke({
    "input": "What is 25 multiplied by 4, then add 100 to the result?"
})

print("\n" + "="*50)
print("FINAL ANSWER:", response['output'])
```

Understanding verbose=True Output

When you run the agent with `verbose=True`, you'll see:

```
> Entering new AgentExecutor chain...

Thought: I need to multiply 25 by 4 first
Action: Multiply
Action Input: 25 * 4
Observation: 100
```

```
Thought: Now I need to add 100 to this result
Action: Add
Action Input: 100 + 100
Observation: 200

Thought: I now know the final answer
Final Answer: 200
```

CHAPTER 5: TYPES OF AGENTS

📋 Agent Types Comparison

LangChain offers different agent types for different scenarios:

```
┌─────────────────────────────────────────────────────┐
│              AGENT TYPES                            │
├─────────────────────────────────────────────────────┤
│ 1. REACT AGENT (Most Popular)                       │
│    • Uses Thought-Action-Observation loop           │
│    • Best for: General purpose tasks                │
│    • Shows reasoning steps                          │
│                                                     │
│ 2. ZERO-SHOT REACT DESCRIPTION                      │
│    • Decides which tool based on descriptions       │
│    • Best for: When you have many tools             │
│    • No prior examples needed                       │
│                                                     │
│ 3. CONVERSATIONAL REACT                             │
│    • Has memory of conversation                     │
│    • Best for: Chat applications                    │
│    • Remembers context                              │
│                                                     │
│ 4. OPENAI FUNCTIONS AGENT                           │
│    • Uses OpenAI's function calling                 │
│    • Best for: OpenAI models only                   │
│    • More reliable tool selection                   │
│                                                     │
│ 5. STRUCTURED CHAT AGENT                            │
│    • Handles complex tool inputs                    │
│    • Best for: Tools needing multiple parameters    │
│    • More structured outputs                        │
└─────────────────────────────────────────────────────┘
```

When to Use Which Agent

Use ReAct Agent (Default choice):

- General purpose tasks
- Need to see reasoning process

- Working with any LLM

Use Conversational Agent:

- Building chatbots
- Multi-turn conversations
- Need to remember context

Use OpenAI Functions Agent:

- Using OpenAI models (GPT-4, GPT-3.5)
- Need more reliable tool calling
- Complex tool interactions

CHAPTER 6: PRE-BUILT TOOLS FROM LANGCHAIN

🔧 Built-in Tools You Can Use

LangChain provides many ready-to-use tools:

EXAMPLE 4: Using Wikipedia and DuckDuckGo Search

```
!pip install -q wikipedia duckduckgo-search

from langchain.tools import WikipediaQueryRun, DuckDuckGoSearchRun
from langchain.utilities import WikipediaAPIWrapper
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
from langchain_google_genai import ChatGoogleGenerativeAI

# Setup LLM
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0)

# Create Wikipedia tool
wikipedia = WikipediaQueryRun(
    api_wrapper=WikipediaAPIWrapper(
        top_k_results=1,
        doc_content_chars_max=500
    )
)

# Create DuckDuckGo search tool
search = DuckDuckGoSearchRun()

# Combine tools
tools = [wikipedia, search]

# Create agent
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(
```

```
    agent=agent,
    tools=tools,
    verbose=True,
    handle_parsing_errors=True
)

# Use the agent
response = agent_executor.invoke({
    "input": "Who is the current CEO of Tesla and what is their latest project?"
})

print("\nFINAL ANSWER:", response['output'])
```

EXAMPLE 5: Python REPL Tool (Run Python Code)

```
from langchain.tools import PythonREPLTool
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
from langchain_google_genai import ChatGoogleGenerativeAI

# Setup
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0)

# Create Python REPL tool
python_repl = PythonREPLTool()

tools = [python_repl]

# Create agent
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    handle_parsing_errors=True
)

# Ask agent to write and run Python code
response = agent_executor.invoke({
    "input": "Create a list of squares of numbers from 1 to 10 and calculate their
sum"
})

print("\nRESULT:", response['output'])
```

CHAPTER 7: MEMORY - Making AI Remember

🧠 What is Memory?

Problem: By default, AI has NO memory. Each conversation is brand new!

Without Memory:

```
You: My name is John
AI: Nice to meet you, John!

You: What's my name?
AI: I don't know your name. 😔
```

With Memory:

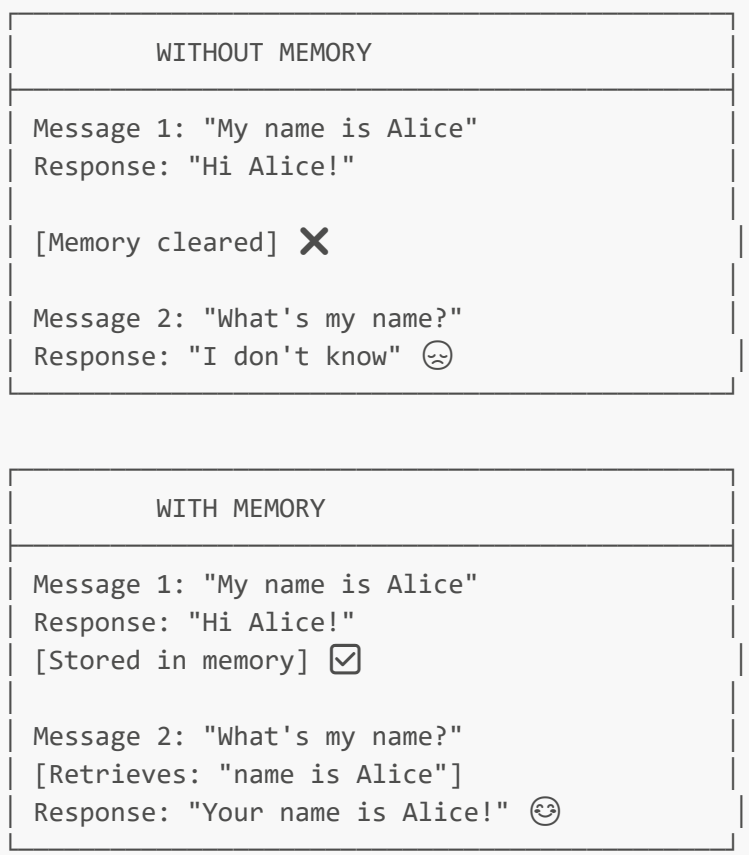
```
You: My name is John
AI: Nice to meet you, John!

You: What's my name?
AI: Your name is John! 😊
```

Memory Types in LangChain

```
┌─────────────────────────────────────────────────────┐
│              MEMORY TYPES                           │
├─────────────────────────────────────────────────────┤
│ 1. ConversationBufferMemory                         │
│    • Stores EVERYTHING                              │
│    • Simple but can get large                       │
│    • Use: Short conversations                       │
│                                                     │
│ 2. ConversationBufferWindowMemory                   │
│    • Stores last N messages                         │
│    • Prevents memory overflow                       │
│    • Use: Long conversations                        │
│                                                     │
│ 3. ConversationSummaryMemory                        │
│    • Stores summary of conversation                 │
│    • Saves tokens                                   │
│    • Use: Very long conversations                   │
│                                                     │
│ 4. ConversationSummaryBufferMemory                  │
│    • Hybrid: Recent messages + old summary          │
│    • Best of both worlds                            │
│    • Use: Production chatbots                       │
│                                                     │
│ 5. ConversationEntityMemory                         │
│    • Remembers specific entities (names, places)    │
│    • Structured memory                              │
│    • Use: Customer service, personalization         │
└─────────────────────────────────────────────────────┘
```

Memory Visual Flow



CHAPTER 8: IMPLEMENTING MEMORY

EXAMPLE 6: ConversationBufferMemory (Basic Memory)

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_google_genai import ChatGoogleGenerativeAI

# Setup LLM
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# Create memory
memory = ConversationBufferMemory()

# Create conversation chain with memory
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True  # Shows what's stored in memory
)
```

```
# Conversation 1
print("="*50)
response1 = conversation.predict(input="Hi! My name is Sarah and I love pizza.")
print("AI:", response1)

# Conversation 2 - AI should remember!
print("\n" + "="*50)
response2 = conversation.predict(input="What's my name?")
print("AI:", response2)

# Conversation 3 - AI should remember!
print("\n" + "="*50)
response3 = conversation.predict(input="What food do I like?")
print("AI:", response3)

# View memory contents
print("\n" + "="*50)
print("MEMORY CONTENTS:")
print(memory.load_memory_variables({}))
```

EXAMPLE 7: ConversationBufferWindowMemory (Last N Messages)

```
from langchain.memory import ConversationBufferWindowMemory
from langchain.chains import ConversationChain
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# Memory that keeps only last 2 exchanges (4 messages)
memory = ConversationBufferWindowMemory(k=2)  # k = number of exchanges to
remember

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# Multiple conversations
print("Message 1:")
conversation.predict(input="My favorite color is blue")

print("\nMessage 2:")
conversation.predict(input="My favorite food is sushi")

print("\nMessage 3:")
conversation.predict(input="My hobby is painting")

# This should remember only the last 2 exchanges
print("\nMessage 4 (Testing memory):")
response = conversation.predict(input="What's my favorite color?")
```

```
print("AI:", response)
# AI might not remember color (too old) but will remember food and hobby!
```

EXAMPLE 8: ConversationSummaryMemory (Summarizes Conversation)

```
from langchain.memory import ConversationSummaryMemory
from langchain.chains import ConversationChain
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# Memory that summarizes instead of storing everything
memory = ConversationSummaryMemory(llm=llm)

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# Long conversation
conversation.predict(input="I work as a software engineer in San Francisco")
conversation.predict(input="I've been learning machine learning for 2 years")
conversation.predict(input="My goal is to build AI products that help people")

# Check summary
print("\n" + "="*50)
print("SUMMARY OF CONVERSATION:")
print(memory.load_memory_variables({}))
```

CHAPTER 9: MEMORY WITH AGENTS

Combining Agents + Memory

Agents with memory can:

- Remember previous questions
- Use context from earlier in conversation
- Provide personalized responses
- Handle multi-step tasks across messages

EXAMPLE 9: Agent with Conversational Memory

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.memory import ConversationBufferMemory
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain import hub
```

```
from langchain.tools import Tool

# Setup
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0)

# Create simple tools
def calculator(expression: str) -> str:
    """Calculates math expressions"""
    try:
        return str(eval(expression))
    except:
        return "Invalid expression"

def save_number(number: str) -> str:
    """Saves a number for later use"""
    return f"Saved number: {number}"

tools = [
    Tool(name="Calculator", func=calculator, description="For math calculations"),
    Tool(name="SaveNumber", func=save_number, description="Saves a number")
]

# Create memory
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Create agent with memory
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True,
    handle_parsing_errors=True
)

# Conversation 1
print("Question 1:")
response1 = agent_executor.invoke({"input": "Calculate 50 * 20"})
print("Answer:", response1['output'])

# Conversation 2 - Agent should remember previous calculation!
print("\n" + "="*50)
print("Question 2:")
response2 = agent_executor.invoke({"input": "What was my previous calculation
result?"})
print("Answer:", response2['output'])
```

CHAPTER 10: COMPLETE PROJECT - Personal AI Assistant

Project Overview

We'll build a complete personal assistant that:

- ☑ Can search the web
- ☑ Can do calculations
- ☑ Remembers conversation context
- ☑ Provides personalized responses

EXAMPLE 10: Full AI Assistant with Memory & Tools

```
!pip install -q langchain langchain-google-genai google-generativeai duckduckgo-
search wikipedia

from langchain.agents import create_react_agent, AgentExecutor
from langchain.memory import ConversationBufferWindowMemory
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain import hub
from langchain.tools import Tool, DuckDuckGoSearchRun, WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper
import os

# Setup
os.environ["GOOGLE_API_KEY"] = "YOUR_API_KEY_HERE"
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# Tool 1: Calculator
def calculator(expression: str) -> str:
    """Performs mathematical calculations"""
    try:
        result = eval(expression)
        return f"Result: {result}"
    except Exception as e:
        return f"Error in calculation: {str(e)}"

# Tool 2: Personal Info Storage (simulated)
user_data = {}

def save_info(info: str) -> str:
    """Saves user information. Format: 'key:value'"""
    try:
        key, value = info.split(':', 1)
        user_data[key.strip()] = value.strip()
        return f"Saved: {key.strip()} = {value.strip()}"
    except:
        return "Please use format: key:value"

def get_info(key: str) -> str:
    """Retrieves saved user information"""
    return user_data.get(key.strip(), f"No information found for '{key}'")
```

```
# Tool 3: Web Search
search = DuckDuckGoSearchRun()

# Tool 4: Wikipedia
wikipedia = WikipediaQueryRun(
    api_wrapper=WikipediaAPIWrapper(top_k_results=1, doc_content_chars_max=300)
)

# Combine all tools
tools = [
    Tool(
        name="Calculator",
        func=calculator,
        description="Useful for mathematical calculations. Input should be a math
expression like '25 * 4 + 10'"
    ),
    Tool(
        name="SaveInfo",
        func=save_info,
        description="Saves personal information about the user. Input format:
'key:value' like 'favorite_color:blue'"
    ),
    Tool(
        name="GetInfo",
        func=get_info,
        description="Retrieves saved personal information. Input should be the key
like 'favorite_color'"
    ),
    Tool(
        name="WebSearch",
        func=search.run,
        description="Searches the web for current information. Use this for news,
facts, or recent events."
    ),
    Tool(
        name="Wikipedia",
        func=wikipedia.run,
        description="Searches Wikipedia for detailed information about topics,
people, or concepts."
    )
]

# Create memory (remembers last 5 exchanges)
memory = ConversationBufferWindowMemory(
    memory_key="chat_history",
    k=5,
    return_messages=True
)

# Create agent
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
```

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True,
    handle_parsing_errors=True,
    max_iterations=5
)

# Function to chat with assistant
def chat(message):
    print(f"\n{'='*60}")
    print(f"YOU: {message}")
    print('='*60)
    response = agent_executor.invoke({"input": message})
    print(f"\nASSISTANT: {response['output']}")
    return response['output']

# Demo conversation
print("🤖 PERSONAL AI ASSISTANT ACTIVATED!")
print("=" * 60)

# Interaction 1: Save personal info
chat("My name is Alex and my favorite programming language is Python")

# Interaction 2: Calculate
chat("What's 15% of 250?")

# Interaction 3: Web search
chat("What's the latest news about AI?")

# Interaction 4
: Test memory
chat("What's my name and favorite language?")

# Interaction 5: Wikipedia search
chat("Tell me about machine learning")
```

CHAPTER 11: ADVANCED MEMORY TECHNIQUES

🎓 Entity Memory (Remember Specific Information)

ConversationEntityMemory remembers specific entities (names, places, objects):

EXAMPLE 11: Entity Memory Implementation

```
from langchain.memory import ConversationEntityMemory
from langchain.chains import ConversationChain
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.7)

# Entity memory - tracks entities mentioned
entity_memory = ConversationEntityMemory(llm=llm)

conversation = ConversationChain(
    llm=llm,
    memory=entity_memory,
    verbose=True
)

# Conversation with multiple entities
conversation.predict(input="My friend John works at Google in California")
conversation.predict(input="My friend Sarah is a doctor in New York")
conversation.predict(input="John loves hiking and Sarah loves reading")

# Test entity recall
response = conversation.predict(input="What do you know about John?")
print("\nAbout John:", response)

response = conversation.predict(input="What do you know about Sarah?")
print("\nAbout Sarah:", response)

# View entity store
print("\n" + "="*50)
print("ENTITY STORE:")
print(entity_memory.entity_store.store)
```

Memory Comparison Table

```
┌──────────────────────┬──────────────┬──────────────┬──────────────┐
│ Memory Type          │ Storage Size │ Use Case     │ Best For     │
├──────────────────────┼──────────────┼──────────────┼──────────────┤
│ Buffer               │ Unlimited    │ Everything   │ Short chats  │
│ BufferWindow         │ Last N       │ Recent       │ Long chats   │
│ Summary              │ Compressed   │ Summary      │ Very long    │
│ SummaryBuffer        │ Hybrid       │ Both         │ Production   │
│ Entity               │ Structured   │ Entities     │ Personal AI  │
└──────────────────────┴──────────────┴──────────────┴──────────────┘
```

CHAPTER 12: AGENT BEST PRACTICES & DEBUGGING

🪝 Common Issues & Solutions

Problem 1: Agent doesn't use tools

```
# ❌ BAD: Vague tool description
Tool(name="Tool1", description="Does stuff", func=my_func)
```

```
# ☑ GOOD: Clear, specific description
Tool(
    name="Calculator",
    description="Use this to perform mathematical calculations. Input should be a
valid Python math expression like '2+2' or '50*0.15'"
)
```

Problem 2: Agent loops forever

```
# ☑ Add max_iterations to prevent infinite loops
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    max_iterations=10,  # Stop after 10 steps
    early_stopping_method="generate"  # Return best attempt if stuck
)
```

Problem 3: Tool errors crash the agent

```
# ☑ Enable error handling
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    handle_parsing_errors=True,  # Catch and handle errors
    return_intermediate_steps=True  # See what went wrong
)
```

Agent Design Tips

```
┌─────────────────────────────────────────────────┐
│         AGENT DESIGN PRINCIPLES                 │
├─────────────────────────────────────────────────┤
│ 1. Start Simple                                 │
│    • Begin with 2-3 tools                       │
│    • Add complexity gradually                   │
│                                                 │
│ 2. Clear Tool Descriptions                      │
│    • Be specific about inputs/outputs           │
│    • Include examples in description            │
│                                                 │
│ 3. Set Temperature = 0                          │
│    • Agents need focus, not creativity          │
│    • Use temp=0 for reliable reasoning          │
│                                                 │
│ 4. Add Safety Limits                            │
│    • max_iterations prevents infinite loops     │
```

```
│    • handle_parsing_errors prevents crashes      │
│                                                  │
│ 5. Use verbose=True During Development           │
│    • See agent's thinking process                │
│    • Identify where things go wrong              │
│                                                  │
│ 6. Test Tools Independently First                │
│    • Make sure each tool works alone             │
│    • Then combine into agent                     │
└──────────────────────────────────────────────────┘
```

CHAPTER 13: REAL-WORLD AGENT PATTERNS

Pattern 1: Research Assistant

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain import hub
from langchain.tools import Tool

llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0)

# Research tools
def search_papers(topic: str) -> str:
    """Simulated academic paper search"""
    return f"Found papers about {topic}: Paper1, Paper2, Paper3"

def summarize_text(text: str) -> str:
    """Summarizes long text"""
    return f"Summary: {text[:100]}..."

tools = [
    Tool(name="SearchPapers", func=search_papers,
        description="Search for academic papers on a topic"),
    Tool(name="Summarize", func=summarize_text,
        description="Summarize long text into key points")
]

prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Use
response = agent_executor.invoke({
    "input": "Research recent papers about transformer models and summarize key
findings"
})
```

Pattern 2: Data Analysis Assistant

```
# Data analysis tools
def load_data(filename: str) -> str:
    """Simulated data loading"""
    return f"Loaded data from {filename}: 1000 rows, 5 columns"

def analyze_data(query: str) -> str:
    """Simulated analysis"""
    return f"Analysis result for '{query}': Mean=50, Median=45, Std=10"

def visualize(chart_type: str) -> str:
    """Simulated visualization"""
    return f"Created {chart_type} chart"

tools = [
    Tool(name="LoadData", func=load_data, description="Load CSV data file"),
    Tool(name="Analyze", func=analyze_data, description="Analyze data with
statistics"),
    Tool(name="Visualize", func=visualize, description="Create charts (bar, line,
scatter)")
]

# Create data analysis agent...
# (Same agent creation pattern as above)
```

Pattern 3: Customer Service Bot

```
# Customer service tools
customer_db = {
    "C001": {"name": "John", "orders": ["#123", "#456"], "status": "active"},
    "C002": {"name": "Sarah", "orders": ["#789"], "status": "premium"}
}

def lookup_customer(customer_id: str) -> str:
    """Look up customer information"""
    info = customer_db.get(customer_id, {})
    return str(info) if info else "Customer not found"

def check_order(order_id: str) -> str:
    """Check order status"""
    return f"Order {order_id}: Shipped, arriving in 2 days"

def create_ticket(issue: str) -> str:
    """Create support ticket"""
    return f"Ticket created: #{issue[:20]}... Priority: Normal"

tools = [
    Tool(name="LookupCustomer", func=lookup_customer,
        description="Look up customer by ID (format: C001)"),
    Tool(name="CheckOrder", func=check_order,
        description="Check order status (format: #123)"),
```

```
    Tool(name="CreateTicket", func=create_ticket,
        description="Create support ticket for issues")
]

# Create customer service agent with conversational memory
# (Combine with ConversationBufferMemory from earlier)
```

CHAPTER 14: KEY CONCEPTS SUMMARY

Complete Concepts Overview

AGENTS

- **What:** AI that thinks, decides, and uses tools
- **Why:** Handle complex, multi-step tasks dynamically
- **When:** Task needs decisions based on intermediate results

REACT PATTERN

- **What:** Reason (think) + Act (do) loop
- **Steps:** Thought → Action → Observation → Repeat
- **Why:** Structured approach to problem-solving

TOOLS

- **What:** Functions agents can use (search, calculate, etc.)
- **Parts:** Name, Description, Function
- **Why:** Extend agent capabilities beyond text generation

MEMORY

- **What:** Allows AI to remember conversation context
- **Types:** Buffer, Window, Summary, Entity
- **Why:** Personalization and context awareness

AGENT TYPES

- **ReAct:** General purpose, shows reasoning
- **Conversational:** Has memory built-in
- **OpenAI Functions:** For OpenAI models, more reliable
- **Structured:** For complex tool inputs

Learning Progression

```
┌─────────────────────────────────────────────┐
│         LEARNING PATH                       │
├─────────────────────────────────────────────┤
│ Week 1: Basics                              │
│  ✓ What are agents                          │
```

```
  │  ✓ Simple tools                                │
  │  ✓ First agent                                 │
  │                                                │
  │ Week 2: Tools & Memory                         │
  │  ✓ Multiple tools                              │
  │  ✓ Pre-built tools                             │
  │  ✓ Basic memory                                │
  │                                                │
  │ Week 3: Advanced                               │
  │  ✓ Conversational agents                       │
  │  ✓ Entity memory                               │
  │  ✓ Error handling                              │
  │                                                │
  │ Week 4: Projects                               │
  │  ✓ Personal assistant                          │
  │  ✓ Research bot                                │
  │  ✓ Customer service                            │
  └────────────────────────────────────────────────┘
```

CHAPTER 15: HANDS-ON PRACTICE EXERCISES

🎯 Practice Project 1: Study Buddy Agent

Build an agent that helps with studying:

```
# Tools needed:
# 1. Quiz generator (creates questions from topic)
# 2. Flashcard creator (makes flashcards)
# 3. Study timer (tracks study sessions)
# 4. Progress tracker (saves scores)

# Your task: Implement these tools and create agent
```

🎯 Practice Project 2: Travel Planner Agent

Build an agent that helps plan trips:

```
# Tools needed:
# 1. Weather lookup (check destination weather)
# 2. Currency converter
# 3. Attraction finder
# 4. Itinerary creator

# Your task: Implement and integrate
```

🎯 Practice Project 3: Code Helper Agent

Build an agent that helps with programming:

```
# Tools needed:
# 1. Code explainer (explains code snippets)
# 2. Error debugger (helps fix errors)
# 3. Documentation searcher
# 4. Code runner (executes safe code)

# Your task: Create comprehensive coding assistant
```

🕮 LEARNING RESOURCES

Official Documentation

- **LangChain Docs:** https://python.langchain.com/docs/
- **Agents Guide:** https://python.langchain.com/docs/modules/agents/
- **Memory Guide:** https://python.langchain.com/docs/modules/memory/
- **Tools Reference:** https://python.langchain.com/docs/integrations/tools/

Video Tutorials

- **LangChain YouTube Channel:** https://www.youtube.com/@LangChain
- **FreeCodeCamp LangChain Course:** Search "LangChain tutorial" on YouTube

Community

- **LangChain Discord:** Join for help and discussions
- **GitHub:** https://github.com/langchain-ai/langchain
- **Twitter:** @LangChainAI for updates

Practice Platforms

- **LangChain Hub:** https://smith.langchain.com/hub (prompt templates)
- **Google Colab:** Free environment to run code
- **Replit:** Online IDE for quick prototyping

☑ FINAL CHECKLIST

Before moving to production:

```
□ Understand agent vs chain difference
□ Can create custom tools
□ Can use pre-built tools (search, wikipedia)
□ Understand ReAct pattern
□ Can implement basic memory
□ Can combine agents + memory
□ Know how to debug agents (verbose=True)
```

```
□ Built at least 2 complete projects
□ Understand when to use which memory type
□ Know agent safety (max_iterations, error handling)
```

🎓 GRADUATION PROJECT

Build a Complete Personal AI Assistant:

Requirements:

1. At least 5 different tools
2. Conversational memory
3. Error handling
4. Can search web
5. Can do calculations
6. Remembers user preferences
7. Provides personalized responses

Bonus Features:

- Save conversation history to file
- Export data to CSV
- Send email summaries
- Schedule reminders

🚀 WHAT'S NEXT?

After mastering agents and memory:

1. **Advanced RAG (Retrieval Augmented Generation)**

- Connect to databases
- Query your own documents
- Build knowledge bases

2. **LangChain Expression Language (LCEL)**

- Advanced chain composition
- Parallel execution
- Custom chains

3. **Production Deployment**

- FastAPI integration
- Streamlit UI
- Docker deployment

4. **Multi-Agent Systems**

- Multiple agents working together
- Agent communication
- Complex workflows

REMEMBER

- **Start Simple:** One tool, then add more
- **Test Often:** Test each component separately
- **Use verbose=True:** See what agents are thinking
- **Handle Errors:** Always include error handling
- **Memory Matters:** Choose right memory type for your use case
- **Practice Daily:** Build small projects regularly

You've learned: ☑ Chains (Week 1) ☑ Agents & Tools (Week 2)
☑ Memory Systems (Week 3)

You're ready to build production AI applications! 🎉

Need help? Reference back to specific chapters!

- Agents: Chapter 1-4
- Tools: Chapter 3, 6
- Memory: Chapter 7-9
- Projects: Chapter 10, 13

Happy Building! 🚀